

Sigil: Universal Hand Gesture Control Framework for Command-Controllable Environments

Prathamesh Sankhe (231080061)

Yadnyesh Patil (231080050)

Advait Desai (241080902)

Raj Kharwal (231080052)

April 2026

1 Problem Statement

Modern workflows across diverse environments—from desktop computing to presentations and virtual collaboration—remain heavily dependent on keyboard and mouse input. This creates friction in scenarios where hands-free control would enhance productivity:

- **Presentations & Teaching:** Instructors need to control slides and multimedia while maintaining audience engagement
- **Virtual Meetings:** Professionals require quick access to mute, reactions, and screen sharing controls
- **Desktop Management:** Linux users running tiling window managers lack intuitive gesture interfaces
- **Media Production:** Content creators need hands-free playback and timeline control

Existing gesture control solutions suffer from: **hardware barriers** (require expensive depth cameras >\$200), **platform limitations** (mobile-only or proprietary systems), **performance issues** (high latency >150ms, CPU usage >50%), and **limited customization** (no user training capabilities).

Core Challenge: Develop a universal gesture control framework enabling sub-80ms latency, $\geq 80\%$ accuracy using only standard webcam, with <25% CPU overhead, adaptable to any command-controllable environment through keyboard shortcuts, shell commands, or APIs.

2 Objective

2.1 Primary Goal

Develop a universal hand gesture control framework that enables users to record, train, and map custom hand gestures (instant, gradual, sequential) to any command-controllable environment—presentations, virtual meetings, window managers, and media applications—with performance suitable for daily use.

2.2 Target Environments & Key Objectives

Environments: Window Managers (Hyprland, Sway, i3), Presentations (Impress, Slides, PowerPoint), Virtual Meetings (Zoom, Meet, Teams), Media Players (VLC, MPV, OBS)

Performance: Latency <80ms, 30-60 fps tracking, CPU <25%, inference <30ms

Accuracy: Landmark detection $\geq 95\%$, gesture classification $\geq 80\%$, zero false positives

Gesture Types: Instant (peace sign → keyboard shortcuts), Gradual (pinch openness → volume control), Sequential (swipe patterns → command chains)

Universal Compatibility: xdotool/xdotool (keyboard), shell commands, window manager IPC (hyprctl/swaymsg/i3-msg), D-Bus/MPRIS (media control), profile auto-switching

3 Methodology

3.1 Architecture Overview

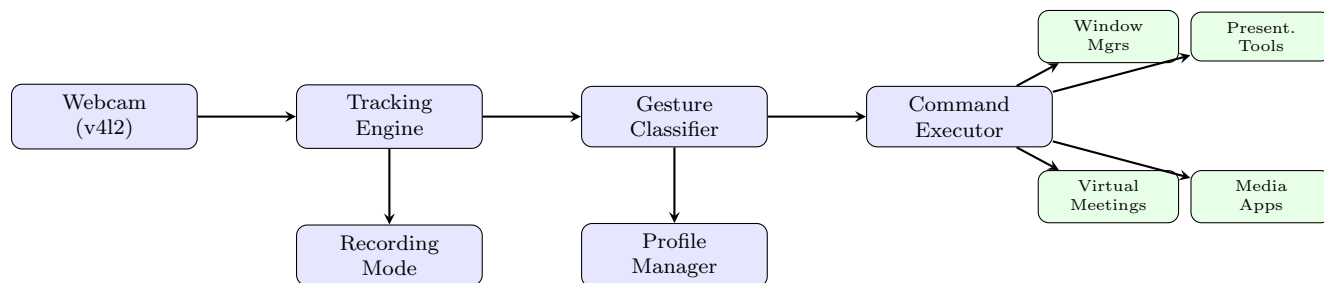


Figure 1: Universal System Architecture

3.2 Phase 1: Tracking Engine

MediaPipe Hand Landmarker extracts **21 3D landmarks per hand** + handedness. Configuration: `num_hands=2, min_detection_confidence=0.7, running_mode=VIDEO`. Input: standard webcam $\geq 720p$ via v4l2/pipewire. Fallback to legacy `mp_hands` if API changes.

3.3 Phase 2: Gesture Classification

Feature Engineering: 101-dimensional vector (normalized 3D positions, finger curl states, inter-landmark distances, joint angles, palm orientation, finger spread)

Models: Instant (MediaPipe Gesture Recognizer .tflite or scikit-learn MLP/KNN), Gradual (geometric features: pinch strength, hand openness), Sequential (time-series buffering 8-30 frames with DTW or LSTM)

3.4 Phase 3: Recording & Training

Trigger: Hotkey Super+Alt+G. Collect 50-200 samples per gesture class. Storage: `~/.local/share/sigil/record`
Training: `sigil train --type instant --class <name>` exports to .tflite model.

3.5 Phase 4: Profile System & Execution

Profiles: desktop.yaml (window controls), presentation.yaml (slides), meeting.yaml (mute/reactions), media.yaml (playback). Auto-switching detects active application.

Configuration Examples:

```

# presentation.yaml
gestures:
  - name: "next_slide"
  
```

```

    action: "xdotool key Right"
- name: "previous_slide"
  action: "xdotool key Left"

# meeting.yaml
gestures:
- name: "toggle_mute"
  action: "xdotool key alt+a"
- name: "raise_hand"
  action: "xdotool key alt+y"

# desktop.yaml
gestures:
- name: "close_window"
  action: "hyprctl dispatch killactive"
- name: "next_workspace"
  action: "hyprctl dispatch workspace +1"

```

Runtime Loop: Capture frame → Detect hands (21 landmarks $\times$ 2) → Classify with confidence → Temporal smoothing (9-frame buffer, 2-frame confirmation) → Match profile mappings → Execute command (keyboard/shell/IPC/D-Bus) → Blanking period (1000ms cooldown)

3.6 Phase 5: Optimization

Model compression (<30ms inference), multi-threading (capture/inference/execution), auto-tuning FPS (30→15 fps fallback based on CPU), logging (debug/info/error to journalctl).

3.7 Technology Stack & Timeline

Component	Technology
Hand tracking	MediaPipe $\geq 0.10.14$
Classifier	Model Maker .tflite / scikit-learn
Camera I/O	OpenCV ≥ 4.9
Config	PyYAML ≥ 6.0
Keyboard sim	xdotool/xdotool
Platform	Linux (X11/Wayland)
Hardware	Webcam $\geq 720p$, AVX2 CPU

Table 1: Technology Stack

3.8 Success Criteria

Functional: Record 3+ gestures (50 samples each), train <60s, profile auto-switching works, commands trigger across environments

Performance: Latency <80ms, CPU <25%, accuracy $\geq 80\%$, zero false positives (5-min idle)

Use Cases:

Repository: <https://github.com/PMS61/sigil>

Milestone	Duration
M1: Tracking + visualization	1-2 weeks
M2: Instant gesture recognition	2-3 weeks
M3: Gradual + sequential	2-3 weeks
M4: Profile system + integration	1-2 weeks
M5: Optimization + packaging	1-2 weeks
Total MVP	9-12 weeks

Table 2: Development Timeline

Environment	Gesture	Action
Presentation	Right swipe	Next slide
Meeting	Peace sign	Toggle mute
Desktop	Close fist	Close window
Media	Thumbs right	Play/pause

Table 3: Multi-Environment Examples

Primary Focus: Initial implementation targets Hyprland on CachyOS/Arch as reference, with architecture designed for universal environment support.